

DEVOPS E ARQUITETURA CLOUD

FASE 2 | AULA 03 -

DOCKERFILE E BUILD DE IMAGENS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS	13

EMANIP

O QUE VEM POR AÍ?

Depois de entender como funcionam os Dockerfiles e de criar as suas próprias imagens, é hora de dar um passo além. Neste trecho da jornada, a gente vai olhar para o que realmente importa quando o assunto é colocar aplicações em produção: **segurança, otimização e confiabilidade**.

Vamos explorar o conceito de multistage builds, aprender a reduzir o tamanho das imagens, evitar riscos desnecessários e escrever Dockerfiles que aguentem o tranco de ambientes reais. Se você já escreveu um Dockerfile antes, mas sentia que dava para melhorar, esta aula vai te mostrar como.

HANDS ON

Chegamos na parte prática da aula. A proposta destes vídeos é justamente mostrar como aplicar boas práticas de Dockerfile voltadas para a produção. Ao longo das videoaulas você verá não só o passo a passo da construção de uma imagem otimizada, mas também como lidar com os desafios reais que surgem no build e na configuração de ambientes mais seguros.

Na primeira aula, vamos focar no conceito de Multistage Builds. Você vai entender como separar o ambiente de build do ambiente de execução, podendo, assim, deixar suas imagens mais leves, rápidas e organizadas. É uma prática muito usada em produção e vamos construí-la juntos, do zero.

Na segunda videoaula, o foco é a segurança. Vamos revisar o uso de imagens base mais enxutas, como alpine, e também boas práticas sobre como não rodar como root e isolar arquivos sensíveis. Tudo isso com exemplos reais e comandos que você pode adaptar para o seu projeto.

Pra fechar, a terceira videoaula é um combo completo: vamos construir, ao vivo, um Dockerfile pronto para produção, aplicando as técnicas de multistage, segurança e também estratégias de troubleshooting. Se algo der errado no meio do caminho (e spoiler: vai dar! rs), você verá como identificar o problema e resolver.

SAIBA MAIS

Imagens base: alpine, slim e distroless

Na hora de escolher uma imagem base, vale pensar com carinho o que realmente é necessário dentro do seu container. Quanto mais leve for a sua imagem, menor será o tempo de build, o tempo de deploy e a superfície de ataque. É aí que entram opções como alpine, slim e distroless.

A alpine é uma distribuição Linux mínima, com cerca de 5MB. Ela já vem empacotada para funcionar em diversos contextos e é compatível com o apk, o gerenciador de pacotes dela. A Alpine é muito usada por quem quer deixar o container o mais enxuto possível, mas ela pode dar um pouco mais de trabalho caso você precise compilar ou instalar bibliotecas que dependam de outras distribuições mais completas.

Já a slim é uma versão reduzida da imagem padrão — por exemplo, node:18-slim — que ainda mantém a compatibilidade com a maioria dos pacotes do Debian, mas corta extras desnecessários. É uma escolha equilibrada pra quem quer reduzir peso, mas ainda precisa de uma base estável.

Por fim, o distroless vai ainda mais longe: ele não tem shell, nem gerenciador de pacotes, nem utilitários do sistema. Só leva o que sua aplicação realmente precisa pra rodar. É excelente pra produção, especialmente do ponto de vista de segurança, mas exige que o build seja muito bem feito e normalmente precisa de multistage builds pra funcionar direito.

Segurança em containers: namespaces, capabilities e seccomp

Quando a gente fala sobre segurança em containers, é fácil cair na armadilha de achar que o Docker já isola tudo e pronto, tá seguro. Mas não é bem assim. O container não é uma VM — ele compartilha o kernel com o host. Isso quer dizer que, se alguém escapar do container, pode ter acesso direto ao sistema. Para evitar esse tipo de problema, o Linux oferece algumas ferramentas poderosas. As principais delas são os namespaces, as capabilities e o seccomp. Vamos por partes.

Começando pelos **namespaces**. Eles são responsáveis por isolar recursos do sistema. Quando você sobe um container, ele ganha um namespace próprio para

várias coisas: processos (PID), sistema de arquivos (mount), rede (net), usuários (user), etc. Isso significa que o que acontece dentro do container não "vaza" — ou pelo menos, não deveria. É por isso que um processo rodando no container só enxerga os outros processos daquele mesmo container, mesmo que na prática ele esteja compartilhando o kernel com outros containers ou com o host.

Já as **capabilities** são um controle mais fino sobre o que um processo pode ou não pode fazer. Por padrão, o root dentro do container não é tão poderoso quanto o root do host — ele vem com várias permissões a menos. Isso é proposital. E você pode customizar isso com a flag `--cap-drop` pra remover capacidades perigosas (como `NET_ADMIN`, `SYS_ADMIN`, etc.), ou `--cap-add` se realmente precisar de alguma. A ideia aqui é seguir o princípio do menor privilégio: dê para o container só o que ele precisa pra funcionar. Nada mais.

E, por fim, tem o **seccomp**, que é um firewall para syscalls. O Linux tem centenas de chamadas de sistema, e o seccomp permite bloquear várias delas com base numa whitelist. O Docker já vem com um perfil padrão que bloqueia chamadas perigosas (como aquelas que permitem montar novos dispositivos ou modificar o sistema de arquivos), mas você pode criar seu próprio perfil personalizado se quiser ser ainda mais restritivo.

Essas três camadas formam a base da segurança de containers no nível do sistema operacional. Elas trabalham juntas para limitar o escopo e o impacto do que roda ali dentro. Mas claro: segurança não é só isso. Uma imagem mal construída ou uma aplicação vulnerável continua sendo um risco. Por isso, o ideal é combinar essas técnicas com outras boas práticas, como não rodar como root, manter dependências atualizadas, usar imagens mínimas e aplicar patches com frequência.

Se você quer colocar containers em produção com confiança, entender esses mecanismos vai te dar muito mais controle e previsibilidade. Vale demais reservar um tempo para estudar cada um com calma e ver como aplicar em seus projetos do dia a dia.

Usuários não root e boas práticas de permissões

Agora vamos falar de um dos tópicos mais importantes — e muitas vezes ignorado — quando a gente começa a colocar containers em produção: rodar como root ou não. Spoiler: a resposta quase sempre é não.

Por padrão, quando você sobe um container, ele roda como root. E o root dentro do container é praticamente o mesmo root do host. Mesmo que ele esteja tecnicamente isolado, isso é um risco enorme. Se alguém conseguir escapar do container (sim, isso pode acontecer), vai ter acesso privilegiado ao seu sistema. E aí, já era.

A primeira boa prática aqui é criar um usuário não privilegiado dentro da imagem, e forçar o container a rodar com ele. Você pode fazer isso com o comando `adduser` (ou `useradd`, dependendo da imagem base) e depois usar `USER` `nomedousuario`. E mais: se os arquivos da aplicação forem copiados para o container, o ideal é já definir o dono desses arquivos com `--chown`.

Por exemplo:

```
FROM node:18-alpine
RUN adduser -D appuser
USER appuser
WORKDIR /app
COPY --chown=appuser . .
RUN npm ci
CMD ["npm", "start"]
```

Comando de Prompt 1 – Definindo o dono dos arquivos com `--chown`
Fonte: elaborado pelo autor (2025)

Aqui você garante que:

- Existe um usuário chamado `appuser` sem privilégios de root;
- O `WORKDIR` está no controle dele;
- Os arquivos foram copiados já com a permissão correta;
- Toda a aplicação roda com permissões limitadas.

Isso impede que qualquer tentativa de escalar privilégios tenha sucesso — o processo dentro do container não tem poder para alterar configurações críticas, montar volumes perigosos, acessar arquivos do host nem modificar a rede.

Outro ponto importante é: evite usar `chmod 777` para resolver problemas de permissão. Isso é como se fosse um “jeitinho”. O ideal é entender quem deve ser dono do quê e garantir que os diretórios e arquivos tenham permissões mínimas necessárias — nada mais.

Se você estiver usando volumes externos (com `docker run -v` ou no `docker-compose`), é bom garantir que o host também permita o acesso do usuário não root. Em alguns casos, pode ser necessário mapear o UID/GID corretos — principalmente se você estiver rodando a aplicação em ambiente Linux mais restrito.

Ou seja: não use root sem necessidade. Criar um usuário próprio, setar permissões corretas e cuidar dos arquivos dentro do container são práticas simples que reduzem MUITO a superfície de ataque.

Lembre-se: um container seguro começa na imagem. E imagem segura começa com um bom Dockerfile.

Dockerignore e build context

Quando você roda um `docker build`, o Docker precisa saber o que ele vai copiar para o contexto de build — basicamente, tudo que está na pasta onde você roda o comando (ou no caminho que você passar).

Isso é o chamado build context. E é aqui que muita gente se atrapalha.

Por padrão, se você não filtrar nada, o Docker vai tentar mandar tudo que está naquela pasta para o processo de build. Isso inclui `.git`, `node_modules`, arquivos grandes, segredos... tudo mesmo. Isso não só deixa o build mais lento, como também pode vazar dados sensíveis para a imagem final.

É aí que entra o `.dockerignore`. Ele funciona como um `.gitignore`, só que para build. Você lista nele o que não deve ser enviado para o build context — simples assim:

```
.git
node_modules
.env
.vscode
*.log
```

Comando de Prompt 2 – `.dockerignore`
Fonte: elaborado pelo autor (2025)

Com isso, o Docker ignora essas pastas e arquivos quando empacota o build context. Resultado? Build mais rápido, imagem menor e mais segura.

Aliás, se o seu build estiver demorando demais ou parecendo pesado demais, confira o que está no contexto. Muita gente manda gigas sem saber, só porque esqueceu de ignorar.

Outra boa prática é sempre rodar o docker build o mais perto possível do Dockerfile, e garantir que sua estrutura de pastas esteja organizada. Isso evita surpresas.

Enfim, usar `.dockerignore` é obrigatório se você quer construir imagens limpas, seguras e eficientes. Não é frescura — é parte do básico bem feito.

Estratégias de cache em builds com npm, pip, etc

Se você já rodou um docker build mais de uma vez, talvez tenha notado: às vezes o build voa, às vezes demora uma eternidade. A diferença quase sempre está no cache.

O Docker armazena o resultado de cada instrução do Dockerfile em camadas. Se ele encontrar uma camada idêntica a uma que já construiu antes, ele reaproveita o cache e segue em frente. Só que para isso acontecer, você precisa saber como escrever o Dockerfile do jeito certo.

Ao longo desta aula, você viu como escrever Dockerfiles mais completos e seguros, aplicando práticas que são comuns em ambientes de produção. Aprendeu a usar multistage builds para reduzir o tamanho das imagens, explorou estratégias de segurança para manter seu ambiente mais protegido e também passou por técnicas de troubleshooting que ajudam bastante quando o build não sai como esperado. Agora é hora de aplicar tudo isso nos seus próprios projetos:

```
COPY package.json package-lock.json ./  
RUN npm ci  
COPY . .
```

Comando de Prompt 2 – `.Copy package. json`
Fonte: elaborado pelo autor (2025)

Por que nessa ordem? Porque o `npm ci` só depende dos arquivos de dependência. Se você copiar o projeto inteiro antes e fizer alguma alteração num arquivo qualquer, o Docker vai invalidar o cache inteiro, incluindo a camada do `npm ci`. Aí, mesmo sem mudar dependência nenhuma, o build baixa tudo de novo — perda de tempo.

A regra é: copie primeiro o mínimo necessário pra instalar as dependências, rode a instalação, e só depois copie o restante do projeto.

Vale a mesma lógica para o pip, yarn, pnpm, composer... qualquer gerenciador de dependências. O que muda são os nomes dos arquivos:

```
# Python
COPY requirements.txt .
RUN pip install -r requirements.txt

# PHP
COPY composer.json composer.lock ./
RUN composer install

# Node com pnpm
COPY pnpm-lock.yaml ./
RUN pnpm install
```

Outra boa prática é usar `--frozen-lockfile` ou `ci` sempre que possível para evitar surpresas e garantir builds mais previsíveis.

Se você quer builds rápidos, confiáveis e sem estresse, pensar no cache não é opcional, é parte do ofício. Escrever Dockerfile é quase uma arte — e o cache bem usado é o seu melhor pincel.

Ferramentas de análise de imagem: Dive e Snyk

Criar imagens pequenas e seguras não é só uma questão de boas práticas: é uma necessidade real quando a gente pensa em produção. E embora escrever um Dockerfile enxuto já ajude bastante, chega uma hora que você precisa ir além do olho e começar a inspecionar a sua imagem de verdade.

É aí que entram ferramentas como o [Dive](#) e o [Snyk](#).

O **Dive** é como uma lupa nas suas camadas. Ele mostra exatamente o que foi adicionado em cada RUN, COPY, ADD — te dá uma visão clara do que está pesando e do que pode ser retirado. Com ele, você consegue ver, por exemplo, que aquele `apt-get install` gigante está enchendo a imagem com vários pacotes que nem são usados na fase final. É muito prático para quem está ajustando o Dockerfile e quer manter o build sob controle.

Já o **Snyk** entra na parte de segurança. Ele escaneia sua imagem em busca de vulnerabilidades conhecidas, usando uma base de dados sempre atualizada.

Instalar uma dependência com CVE grave? O Snyk vai apontar. Rodar com imagem base desatualizada? Ele avisa também. E o melhor, funciona via CLI, fácil de integrar em pipelines, e também tem um dashboard bonito se você estiver usando junto com o time.

Ambos cumprem papéis diferentes, mas se complementam muito bem: o Dive te ajuda a entender o tamanho da imagem, o Snyk cuida da segurança. Juntos, eles deixam seu container mais enxuto, confiável e pronto para a produção.

Se você ainda não usou nenhum dos dois, vale parar uns minutos e experimentar. É aquele tipo de ferramenta que, depois de usar uma vez, vira parte do seu processo para sempre.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, vimos como escolher imagens base adequadas (alpine, slim e distroless) para otimizar tamanho, desempenho e segurança, além de práticas essenciais como uso de `.dockerignore` para reduzir o contexto de build e estratégias de cache com gerenciadores de dependências (npm, pip, composer, etc.) para acelerar builds. Também exploramos o uso de multistage builds para criar imagens mais leves e a importância de ferramentas como Dive e Snyk para análise de camadas e identificação de vulnerabilidades.

No aspecto de segurança, aprendemos como namespaces, capabilities e seccomp reforçam o isolamento dos containers e como rodar aplicações com usuários não root reduz riscos de escalonamento de privilégios. Ajustar permissões corretamente, manter imagens enxutas e aplicar boas práticas no Dockerfile são medidas que fortalecem a proteção em produção. Assim, unir eficiência no build com foco em segurança garante imagens mais confiáveis e preparadas para um uso real.

REFERÊNCIAS

DOCKER INC. **Dockerfile reference**. 2025. Disponível em: <https://docs.docker.com/engine/> . Acesso em: 20 ago. 2025.

HIGHTOWER, K.; BURNS, B.; BEDA, J. **Kubernetes: Up and Running**. 2. ed. Beijing: O'Reilly Media, 2019.

MERKEL, D. **Docker: lightweight Linux containers for consistent development and deployment**. Linux Journal. Edição 239, 2014. Disponível em: <https://linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>. Acesso em: 20 ago. 2025.

PALAVRAS-CHAVE

Docker. Containerização. Segurança da informação.

EMSE



POSTECH

EMSE